When Does a Neural Architecture Earn Its Cost?

Oruži Zarathustra Goertzel

July 2026 — working note

Abstract

Architecture choices are usually settled by benchmark. We ask a different question: under what conditions does a component *provably* earn, or fail to earn, its cost? We treat four design decisions as independent axes rather than competitors—how belief is represented, where state lives, how a coupled constraint is solved, and how parameters change—and give machine-checked boundaries for each. Fusion of independent evidence is addition in natural coordinates, and every hand-designed gating rule is that addition seen in another chart; the four ways this stops being optimal each have a closed form. Typed slots are licensed exactly when cross-slot coupling vanishes. Command routing collapses to a single gated interpolation, so it buys selectivity rather than a new update class, and order-independence of routed phases holds exactly when generators commute and an affine obstruction vanishes. Five named predictive-coding mechanisms have constructive backpropagation realizations; the residue is an execution-model property, not a training-rule advantage. Forgetting decomposes into a metric part and a connection part, and one Gram matrix certifies both a curriculum's right to be reordered and a command schedule's right to be treated as a set. Finally, in a *linear-effect* model—one that concrete linear attention and additive evidence registers provably instantiate, and that softmax provably breaks—consolidating fast state into weights is faithful exactly when the fast effect is weight-reachable; bounded capacity forces a computable consolidation deadline, and the cadence objective derived from the model's own accounting puts the optimal consolidation at that deadline, refuting the interior optimum a posited quadratic had suggested. We are explicit throughout about which results are theorems about a declared model and which quantities remain empirical; a closing section states what is assumed rather than proved, and what would discharge it. The axes are then assembled, without identifying them, in a *unified carrier*: recurrent control, typed workspace contents, belief metadata and routed operators inhabit one product state whose workspace endpoint and strict non-workspace witnesses are both machine-checked.

Open artifacts. The formal repository, paper source, and compiled PDF are public. The source links below are immutable permalinks to commit `84ec10fdac3c576b725133a70b6e41f45d1874f4`.

1 Introduction

Debates about neural architecture are usually resolved by running two systems and comparing a number. That settles which system won a benchmark; it does not say why, and it does not transfer. This note collects a different kind of result: for each of a small number of design decisions, a condition under which the decision is provably right, and a condition—usually a fixture, often a counterexample—under which it is provably wrong.

Two commitments shape everything that follows. First, the models are exact and small: scalar and matrix linear-Gaussian systems, quadratic tasks, finite regime families. The purpose is to locate boundaries, not to predict benchmark scores, and we mark the places where a boundary is a theorem and the places where the corresponding quantity has to be measured. Second, every claim below is machine-checked in a Lean 4 development, and the negative results are checked with the same seriousness as the positive ones. Several of the sharpest statements here are refutations of things we previously believed.

The organizing idea is that the usual contest—predictive coding *versus* backpropagation, workspace *versus* recurrence, learned mixing *versus* Bayesian fusion—is a category error. These are independent axes that compose. A system chooses, separately: how accumulated belief is represented, where state lives and how it is addressed, how a coupled constraint is solved at inference time, and how parameters are updated. Asking which axis "wins" has no answer; asking when each earns its cost does. No processor designer asks whether registers beat RAM, or whether the cache-coherence protocol beats the arithmetic unit; each component answers a different question, and the architecture is the discipline of their composition. The results below are the analogous discipline for learned machines: not a tournament, but a datasheet of conditions.

2 Four axes, not four contestants

We fix the decomposition used throughout:

axis	the decision
belief coordinates	point estimate, natural coordinates, or a mixture
state carrier	recurrent control, typed slots, routed operators, or their product
inference solver	direct evaluation, or an iterative relaxation to equilibrium
plasticity rule	backpropagation, or a local energy-based update

The axes are genuinely independent: a typed-slot workspace can carry point estimates and train by backpropagation, and a single recurrent vector can carry natural coordinates and settle iteratively. The formal development reflects this—the state theorems do not mention the plasticity rule, and the plasticity theorems do not mention the state carrier.

One consequence is worth stating early, because it is easy to hope otherwise. Within the finite family of regimes we model, no single recommendation is correct everywhere: for each candidate rule there is a modelled regime in which it fails its own guarantee (`no_singleRecommendation_covers_modeledFamily`). The proof is a case analysis that exhibits, for each rule, a concrete regime defeating it. There is no universal architecture, and that is a theorem rather than a hedge.

3 The belief axis: fusion is addition

Write belief in *natural coordinates*: evidence counts (n^+, n^-) in the discrete case, the information form $(\eta, \Lambda) = (\Sigma^{-1}\mu, \Sigma^{-1})$ in the Gaussian case. Then Bayesian fusion of independent evidence is *addition*, and the familiar update rules are that addition seen in another chart.

Theorem 3.1 (Fusion is addition; charts are shadows)**.** *Gaussian fusion, probabilistic-logic revision in weight-of-evidence form, and revision on binary counts are the same operation (`gaussianFusion_eq_plnRevisionWTV, gaussianFusion_eq_binaryCounts_revision_strength`). The settled state of a linear-Gaussian relaxation is the posterior mean (`equilibrium_iff_eq_posteriorMean`), so "settling performs inference" is a theorem rather than a slogan.*

The practical reading is that the optimal gate in a gated-interpolation cell is the Kalman gain: a learned mixer can at best match hardwired addition when the measurement model is calibrated, never beat it. Learning is therefore necessary in the *measurement* maps and the *gain schedule*, not in the fusion step.

That optimality is conditional, and each condition fails somewhere real. Addition never forgets, so under drift it is provably suboptimal and the correct old-information retention is the derived rate $P/(P+Q)$ rather than a tuned constant. Naive addition double-counts correlated re-observation, and the overstatement is exactly the declared overlap. A single natural-coordinate slot projects away multimodal posteriors. And a miscalibrated measurement model poisons exact fusion, though recalibrating the measurement map reproduces the optimal update with fusion left hardwired—so every fusion-stage repair relocates to the measurement map. These four boundaries are the entire content of "when should the belief axis stop being additive".

4 The state axis: typed slots, routing, and what stays safe

Moving from one recurrent vector to addressed slots adds structure and gives away freedom. The licensing condition is sharp: independent per-slot treatment is correct exactly when the cross-slot coupling vanishes, and coupled slots require a joint correction rather than a per-slot approximation.

A command-routed workspace—opaque control tokens selecting a soft mixture of reusable read–transform–gate–write operators—sits on the same axis. Three results delimit it.

Theorem 4.1 (Routing buys selectivity, not a new update class)**.** *A simplex mixture of gated-interpolation experts is again a single gated interpolation under the mixed gain (*`mixedExpertStep_eq_singleInterpolation`*). Command tokens carry no semantics of their own: trajectories are determined by the induced routing schedule alone (*`commandTrajectory_eq_of_mixtureSchedule_eq`*). Immutable per-slot evidence survives every routing schedule, temperature and depth (*`runTripleSlotSchedule_evidence`*).*

Theorem 4.2 (Compositionality is commutation)**.** *Swapping two affine phases shifts the state by* $[B, A]x + (Ba + b - Ab - a)$ *(*`affineCompositionDiscrepancy_exact`*); order-independence holds if and only if the linear parts commute* and *the affine obstruction vanishes (*`affinePhases_orderIndependent_iff`*). Commuting linear parts alone do not suffice—a one-dimensional bias counterexample closes that loophole. Equivalently the pairwise interference energy* $\|[A, B]\|_F^2$ *must vanish (*`linearPhases_orderIndependent_iff_interferenceEnergy_zero`*).*

Theorem 4.3 (Stability does not compose)**.** *Two commands that each individually extinguish in two steps can alternate to* 4^n *growth (*`individuallyStable_switchingDiverges`*). Per-command guarantees are therefore worthless for command* programs*; the honest certificate is schedule-uniform, and commuting stable generators always admit a common quadratic Lyapunov function (*`commutingCommonQuadraticLyapunov_crown`*).*

Against all of this, safety is indifferent. For arbitrary routes, temperatures, schedules and depths, ranked acceptance coincides with checker acceptance and recall of accepted outputs is unchanged (`inheritanceCrown`). When legality is owned by an external checker and the network only orders legal choices, the entire state axis lives on the safe side by construction, and the theorems above are about competence rather than safety.

For readers arriving from the transformer literature, the whole state axis is a familiar machine with one substitution. Attention is the *read* at every point—the encoder, the state carriers, the action scorer—and what varies across the axis is only the *write*:

mechanism	transformer	CAROM	this work
read	multi-head attention	content-addressed read	same, at three sites
transform	feed-forward block	per-operator ϕ_k	per-operator maps
write	residual + norm	gated residual	gate / residual / *addition*
state	residual stream, KV cache	fixed-address slots	typed slots or (n^+, n^-)
routing	mixture-of-experts gate	command-conditioned mix	selectivity, same class
recurrence	weight-tied loop	T recurrent updates	settling, interior-optimum depth
no-weight adaptation	in-context learning	workspace contents	evidence registers, exact

The last column's entries are not analogies but theorems: routing stays in the gated-interpolation class, the belief write is exactly Bayesian fusion in natural coordinates, the recurrence depth has a proved interior optimum, and the evidence registers are an exact instance of the fast-state model of §9—the one member of the test-time-adaptation family whose calibration is provable rather than empirical.

5 The unified carrier: one machine, not four rivals

The axis decomposition is not an instruction to keep four separate decoders forever. Ask what each component actually is, in the vocabulary of a machine. The recurrent vector is a control register: cheap, sequential, decisive. The typed workspace is addressed memory: contents at fixed, elaborator-owned locations. The belief registers are the memory's *metadata*: how much evidence stands behind each cell, and therefore how fast it should be overwritten — the role error-correction bits and replacement policies play in hardware. The routed operators are a small shared arithmetic unit. A hardware designer would never run a tournament among these; the interesting object is the machine that has all of them, each doing its own job. The unified carrier places them in one product state.

For each typed slot i, let

$$(w_i, n_i^+, n_i^-, u_i^+, u_i^-, t_i)$$

denote content, posterior evidence, episode-local innovations and an elaborator-owned type signature. A compact recurrent state h carries chronology. Writing the featurewise precision vector $\Lambda_i = n_i^+ + n_i^-$ and denoting its feature mean by an overbar, the derived retention vector and scalar content gain are

$$r_i = \frac{1}{1 + \sigma_{\text{jump}}^2 \Lambda_i}, \qquad \gamma_i = \frac{\bar{\Lambda}_i^{\text{fresh}}}{\overline{r_i \odot \Lambda_i} + \bar{\Lambda}_i^{\text{fresh}}}.$$

Operator k proposes a candidate c_{ki}, learned gate g_{ki} and nonnegative fresh evidence packet. One settle step is

$$n_i^\pm \leftarrow r_i n_i^\pm + \text{fresh}_i^\pm, \qquad u_i^\pm \leftarrow u_i^\pm + \text{fresh}_i^\pm,$$
$$w_i \leftarrow w_i + \frac{1}{K} \sum_k \gamma_i g_{ki} (c_{ki} - w_i), \qquad h \leftarrow \text{GRU}(x_{\text{carrier}}, h).$$

Thus the learned workspace gate chooses *what* to write while the precision ratio prices how much the current evidence licenses it. When old precision is zero and fresh precision is positive, $\gamma_i = 1$ and the content update is exactly the workspace write (`contentStep_zeroOld_eq_workspace`). When both are zero, the implementation-defined gain is zero; this boundary prevents a division-by-zero "workspace collapse" from being asserted without evidence.

Precision-routed reads. For operator k, slot j receives

$$\ell_{kj} = \frac{\langle q_k, k_j \rangle}{\sqrt{d}} + \beta_k \log(1 + \bar{\Lambda}_j), \qquad \beta_k = b_k^2 \geq 0.$$

The exact identity

$$e^{\ell_{kj}} = e^{\langle q_k, k_j \rangle / \sqrt{d}} (1 + \bar{\Lambda}_j)^{\beta_k}$$

is proved in `exp_precisionBiasedLogit`; positive β_k makes pairwise odds strictly increase with precision, while $\beta_k = 0$ recovers the workspace read. This is an algebraic reliability bias. It is not called a posterior-predictive distribution without separately declaring a probabilistic model.

Persistent evidence without double counting. A fresh hole may initialize its evidence from a persistent table keyed by the injective tuple (role, parent operator, argument). At an episode boundary the table decays once and adds only the innovations $u^{\pm}$ produced during that episode. Reabsorbing the posterior $n^{\pm}$ would import the prior a second time; the development proves that error on a concrete fixture (`posterior_reabsorption_doubleCounts_fixture`). Batch absorption is intentionally outside the causal interface because episode order changes the decay. The persistent feature is disabled by default until an experiment declares how episodes are ordered.

Refinement is exact and nontrivial. Under an explicit mode, parameter relation and state projection, the unified carrier simulates the typed workspace for every finite policy trajectory (`workspaceEndpoint_policyTrajectories_eq`). This is not validation by definition: positive old precision separates the full carrier from that endpoint (`positivePrecision_fullCarrier_separates_workspace`), and two equal workspace/evidence snapshots with different recurrent histories can produce different policies (`recurrentHistory_fullPolicy_separates`). The common masked source policy remains supported exactly at legal operators, with an illegal-operator fixture (`illegalOperator_stays_absent_after_unifiedStep`). The carrier may reorder legal actions; it cannot widen the language.

When does the metadata plane earn its cost? In the scalar linear-Gaussian jump model, the exact one-revision risk reduction is $R^2/(P+Q+R)$ and decreases strictly with jump variance. Including a one-time activation cost and recurring metadata cost yields a decidable revision-count threshold (`retentionNetAdvantage_pos_iff_revisionThreshold`). The same file proves a low-drift positive fixture and a high-drift negative fixture. This says what the architecture experiment must measure—revision count, drift and work in common cost units—rather than predicting that persistent belief will win.

Executable boundary. The exact-real transition has a recorded addressed-packet witness whose child order, type indices, binary32 words and nonzero fresh packet are checked together (`productionInvocation_certificate`). A separate decision-site fixture decodes real binary32 weights and observations and proves all coordinate errors within their budgets (`certificateBatch_total_error_is_bounded`). These are authenticated pointwise witnesses, not a claim that every tensor operation in a training run has been reconstructed in the kernel.

The unified carrier is therefore an architecture specification and a verified refinement, not an efficacy result. Its decisive experiments remain single-variable: recurrent control on/off; precision-read bias on/off; Bayes gate versus unit gain; persistent prior on/off; and the carrier crossed with a mechanism-distinct credit rule. All share the unchanged typed-action waist and external checker.

6 The plasticity axis: what of predictive coding is a backprop trick

Predictive coding is often presented as a rival to backpropagation. Within the finite, declared family of mechanisms we model, most of it is a preconditioner.

Theorem 6.1 (Subsumption partition)**.** *Of six named mechanisms, five have constructive backpropagation realizations—an explicit preconditioned or proximal update reaching the same endpoint or bound—and the sixth, synchronized local parallelism, is an* execution-model *property rather than a training-rule advantage (*`bp_subsumption_partition_crown`*). Unit-rate error-coordinate predictive coding simply is the backpropagation product (*`identityPreconditionedPC_bpEquivalentLicense, epcOneStep_bpEquivalentLicense`*), and curvature-normalized preconditioning is loss-decreasing exactly on the open interval* $(0, 2)$*, with unit rate the one-step optimum.*

Two caveats are load-bearing. Larger negative curvature is not an escape algorithm: at an exact saddle a preconditioned gradient step still has zero gradient, so escape requires a perturbation or an explicit negative-curvature step, and the partition states this rather than eliding it. And the execution-model residue is not a free win either—on a unit-bandwidth synchronous chain, exact predictive coding has no strict latency advantage (`exactUnitBandwidthPC_no_strictDigitalLatencyAdvantage`), while propagation remains bounded by topology distance and local bandwidth (`pcFiniteSpeed_distance_le_sweeps_mul_bandwidth`). The genuine residue is the serial-versus-parallel gap (`synchronizedLocalCost_serial_parallel_exact`), which is a statement about hardware, not about learning rules.

The continual-learning claim deserves the same treatment. Under matched target progress, restriction, exact scheduling and vanishing interference, a predictive-coding adapter can still forget *more* than a backpropagation adapter—the two solutions are reflections across the optimum (`frozenAdapter_four_hypotheses_insufficient`). The advantage is restored only by a fifth condition requiring both residuals to share a sign (`frozenAdapter_five_hypotheses_restore_sourceForgetting_bound`). That condition is a controlled-adaptation assumption, and uncontrolled online data is precisely where it fails.

7 Depth is repairable; cost is not

Iterative relaxation degrades badly with depth, and the degradation has a diagnosis: the per-layer precision-weighted energy is imbalanced across the stack, and a stability-bounded activity rate slows the error front so that layer l is reached only after $L - l$ sweeps. Recent work repairs this to accuracy comparable with backpropagation on deep residual networks, and we have formalized the mechanisms at arbitrary depth over full covariance matrices with a nonlinear Jacobian remainder: the first-arrival attenuation that produces the imbalance, a precision spike that exactly cancels it, a forward-reference update that bounds deep-layer accumulation, auxiliary buffers that synchronize residual and main paths, and the invariance of frozen normalization statistics (`vectorDepthRepair_crown`, `depthScalingRepair_crown`).

The scope should be read narrowly. These are exact local identities and propagation bounds. Convergence and accuracy for trained deep networks are classified in the development as empirical targets, not theorems. And no repair touches the price: the repaired dynamics remain an iterative relaxation, so "matches backpropagation" continues to mean matches its target, not its cost (`pcBPCompute_crown`).

8 Forgetting has a metric side and a connection side

Sequential learning departs from additive learning in two independent ways. For two quadratic tasks with curvatures A_1, A_2, step size η and gradient g_1,

$$\theta_{1\to 2} - \theta_{\text{add}} = \eta^2 A_2 g_1(\theta), \qquad \theta_{1\to 2} - \theta_{2\to 1} = \eta^2 [A_1, A_2]\theta.$$

The first is a *metric* effect—double counting, present even when the order cannot matter. The second is a *connection* effect: order matters exactly to the extent that curvatures fail to commute. A rank-one trichotomy separates them cleanly: parallel tasks commute and any forgetting is pure double counting; orthogonal tasks do not interact; oblique tasks carry the irreducible commutator (`rankOne_parallel_orthogonal_oblique_trichotomy`). Per-pair conflict is measured exactly by the diagonal of an interference Gram built from commutators, which vanishes iff the pair commutes (`pairwiseInterferenceEnergy_eq_zero_iff_commute`).

Two natural-looking claims are false, and the counterexamples are machine-checked. A trivial rotational factor does *not* imply commutation in general: a palindromic product ABA of symmetric matrices stays symmetric when $[A, B] \neq 0$. And interference *spectra* transfer under any similarity (`matrixCommutator_charpoly_unitConjugate`) while interference *angles* transfer only under orthogonal conjugation—an explicit shear breaks them. Pointwise agreement of values and gradients does not determine curvature at all (`pointwiseAgreement_not_determineHessian_negativeExample`), so behavioural similarity is never sufficient to trust a transferred measurement.

The unifying observation is that one Gram matrix does double duty: it certifies both a curriculum's right to be reordered and a routed command schedule's right to be treated as a set (§4). The continual-learning side and the architecture side are the same fact seen from two directions.

9 Fast state versus weights

A system may adapt at inference time in bounded fast state—a context, a cache, a bank of evidence registers—over frozen parameters, and periodically consolidate that state into weights. This is the deployment question behind personalization: how much adaptation belongs in state, and how often, if ever, must weights move?

The results in this section are theorems about a linear-effect model: the readout is the sum of a slow effect and a fast effect, each a linear map of its argument. We state the commitment here rather than in a limitation, because it is doing the work.

Theorem 9.1 (Faithfulness is reachability)**.** *In a linear-effect model, consolidating fast state into weights preserves the adapted readout exactly when the slow update reproduces the fast effect (`faithfulConsolidation_iff_effect_eq`); some faithful consolidation exists exactly when the fast effect lies in the range of the weight-update map (`exists_faithfulConsolidation_iff_mem_range`). A transverse fixture outside that range has an exact irreducible consolidation loss.*

Theorem 9.2 (Capacity forces a deadline)**.** *Under constant arrivals at rate r into capacity C, all evidence is admitted before the first saturation time $\lceil C/r \rceil$, and from that point admitted evidence is exactly C while loss is exactly $rt - C$. The deadline is computed, not chosen.*

Three further boundaries follow. Consolidation double-counts unless the reused contribution is discounted: carrying it at full weight yields exactly one extra copy in both precision and natural-parameter coordinates, and zeroing that contribution restores once-each calibration (`reused_contribution_exact_excess`). The order of consolidations matters exactly when their curvatures fail to

commute—the same Gram criterion as §8—so with oblique periods, permuting two of them yields a different final model. And in a stationary, within-capacity, fast-reachable regime, pure fast-state adaptation attains zero prediction error while any equally faithful consolidation attains the same error at strictly greater cost; consolidation becomes necessary exactly when the weights can realize the shift *and* either capacity is exceeded or the shift leaves fast-state reachability (`twoTimescaleAdaptation_crown`).

Two families of practice bracket this section. On the weight side, constrained-update continual learning already builds the projections our faithfulness condition characterises: episodic-memory methods constrain an update against remembered losses [1], and orthogonal-gradient methods project into a subspace leaving earlier outputs unchanged [2]—both are instances of the minimum-change update our reachability criterion licenses, and both inherit its incompatibility boundary. On the state side, a growing family makes the fast state itself the adapting object: test-time-training layers treat the hidden state as a model updated by self-supervised learning [3], one recurrence is derived directly from implicit online regression [4], and gated delta rules add learned erasure to targeted writes [5]. Our additive evidence registers are the conservative, provably calibrated member of that family; the gated variants trade exact fusion for the forgetting a drifting world requires, which is precisely the boundary of Section 3.

That last statement is the one with practical force: it says weight updates are not the default, they are what you resort to when bounded state provably cannot carry the adaptation. How often to consolidate inside the feasible band is now derived rather than declared: tracing the objective to the development's own accounting—a counted update cost falling like $1/T$ against the hinge of lost evidence, identically zero before the saturation deadline—the unique positive-period optimum is the *deadline itself* whenever losing a full register costs more than an update (`derivedCadenceObjective_at_exactDeadline`). An earlier draft of this theory optimized a posited symmetric quadratic instead; it selects an interior period, disagrees with the derived objective by an exact computed gap on the same fixture, and survives only as a declared approximation. Consolidate at the deadline the capacity computes, not on the schedule a smooth objective manufactures.

10 Search under a budget, and the corpus it produces

A discovery system spends a fixed budget of draws from a learned prior and keeps what an external checker verifies. Three measured facts from our own campaigns had no theory until recently: roughly three quarters of all draws were duplicates of programs already proposed; the four architectures' discoveries were 8–16% overlapping, with about 70% of the union found by exactly one architecture; and a fixed-split portfolio underperformed its own best member. Each is now a theorem about the draw model rather than an anecdote.

Expected *distinct* coverage under n independent draws is $\sum_t (1 - (1 - p_t)^n)$. The exact identity is `iidExpectedDistinctCoverage_eq_sum_one_sub_pow`; the duplicate burden is exactly expected hits minus expected distinct coverage (`iidExpectedDuplicateBurden_eq_hits_sub_distinct`); the two-draw collision probability is the prior's collision mass (`twoDrawCollisionProbability_eq_collisionMass`), which is what a 75%-duplicate search is measuring about its own prior. Deduplicating draws by a semantics-preserving canonical form buys back exactly the collapsed collision mass—the formal price tag on canonicalization, computed before building it. Sharpness has an interior optimum determined by ranking quality, not entropy alone, and two priors with equal entropy and different gold mass have strictly ordered yield: decisiveness is not a dial that buys ranking. The portfolio results close the loop (`budgetedSearchYield_crown`): equal standalone yields can carry radically different union value through complementarity, and a fixed split can lose to the best single arm while an adaptive

allocation dominates both—which is the observed 399-versus-435 in one line.

What the search produces is a corpus, and the corpus is now a formal object of its own: a provenance-authenticated many-to-many relation between programs and the sequences they solve, preserving every distinct program witness rather than quotienting programs by what they solve (`provenanceAwareProgramDiscovery_crown`). Its equivalence ladder—exact syntax, finite-prefix agreement, finite-corpus footprint, full extensional equality—is kept honest by a counterexample at each invalid converse, including a finite-probe rigidity witness evaluated through the real synthesis semantics: agreement on a probed prefix does not force extensional equality. The dependence-aware evidence bridge enforces the matching discipline on statistics: witness multiplicity is not independent evidence, exact reuse receives overlap correction, and descendants trained on earlier discoveries do not count as fresh confirmation. Partial agreement can guide search without touching soundness: ranking by first-mismatch depth can never create an acceptance (`partialRanking_cannot_create_acceptance`), and a two-queue scheduler that reserves a baseline fraction bounds how much a misguided heuristic can delay any baseline discovery (`baseline_rank_time_slowdown_bound`)—so semantic shaping is admissible as guidance precisely because it is powerless over acceptance.

11 What the telemetry can and cannot see

A map is only useful if you can locate yourself on it. Each boundary above is stated in terms of regime parameters—overlap, drift, distortion, cross-slot coupling, commutator energy—and the natural question is whether those are recoverable from what a deployed cell actually logs.

Mostly they are not. Of eleven diagnostics, two are recoverable from primitive telemetry: gain variation from two logged gains, and the scalar spectral envelope from the settling residual. The other nine are *confounded*: for each, two admissible regimes emit identical primitive telemetry and differ in the diagnostic. Each confound comes with the named intervention that resolves it—innovation-residual series for drift, source-identity metadata for overlap, calibration pairs for distortion, paired counterfactual endpoints for the plasticity branch, cross-slot perturbations for coupling, paired operator Jacobians for commutator energy, and Lyapunov measurements for switched stability.

This has a sharp consequence for interpreting experiments. The very quantity that would explain a typed-slot advantage—the cross-slot coupling—is provably not identifiable from ordinary logs (`crossSlotHessian_not_identifiable`), so an observed workspace win cannot be attributed to the mechanism the theory names without adding the perturbation probe. Attribution requires intervention, and the required interventions are enumerated rather than guessed. A search over probe subsets yields exactly two inclusion-minimal suites, differing only in how routed behaviour is identified, and an adaptive procedure that stops at the first recommendation-determining prefix.

12 Scope, and what is assumed rather than proved

We close with the boundary of the boundary theory, stated plainly because several results above are easier than they look.

Typed linearity, now discharged where it can be. The two-timescale results of §9 hold in a model whose effects are linear maps, and that commitment initially carried the argument invisibly: faithfulness reduces to cancellation and reachability to the definition of an image, with no hypothesis to flag. The boundary is now a theorem rather than a hope. Concrete linear attention—outer-product accumulation read by query contraction—provably *instantiates* the model, with additivity over context concatenation derived before anything is packaged as a linear map (`linearAttentionContextEffect_`

append); the belief cell's evidence registers instantiate it exactly; and the abstract range condition becomes a concrete rank condition—reachability through an r-dimensional adapter bottleneck holds iff the required update has rank at most r (rankBudgetReachable_iff_rank_le), so a rank-one adapter facing a rank-two context effect provably cannot consolidate faithfully. Softmax attention falls outside: normalization couples the context and breaks exact additivity, with a two-token counterexample and a quantitative defect bound (scalarSoftmaxAttention_not_additive_fixture, abs_normalizedContextMerge_sub_add_le). What remains empirical is whether a trained transformer's in-context adaptation operates in the near-linear regime—a question on which the evidence is suggestive rather than settled: linear attention is exactly the kernelised recurrence whose state is an accumulated outer product [6], in-context learning has been read as an implicit gradient step [7, 8], and empirically a demonstration set's effect can be compressed into a single added vector [9, 10], which is the additive-effect hypothesis in its most directly testable form. None of that is a proof, and we do not treat it as one.

Derived, not declared, cost models. An earlier draft optimized the consolidation cadence against a posited quadratic trade-off. Deriving the objective from the development's own accounting refuted it—the honest optimum is the corner of §9—and the episode is retained deliberately: a cost model written down rather than derived is an assumption wearing a formula's clothes.

Model families. Except where noted, the results are scalar or matrix linear-Gaussian and quadratic. The depth-repair development reaches full covariance and a nonlinear Jacobian remainder but still proves exact local identities rather than convergence. Trained nonlinear accuracy is an empirical target everywhere it appears.

Negative results are load-bearing. Several of the most useful statements here are refutations—no universal recommendation, stability does not compose, four hypotheses do not suffice for a continual-learning advantage, commuting linear parts do not imply order-independence, nine diagnostics are not identifiable, softmax is not additive. We regard the counterexamples as the more durable half of this work, and the stance has already paid within this note's own lifetime: the first cadence formula did not survive its honest derivation, exactly as anticipated.

Provenance

The development is a Lean 4 library. Definitions, theorem statements and complete proofs are available under the machine-learning tree of the immutable formal snapshot. The unified-carrier development, credit-transport development, utilization boundaries, subsumption partition and two-timescale theory are included in that snapshot. Crown theorems depend only on the standard logical basis (`propext`, `Classical.choice`, `Quot.sound`); the development contains no declared axioms and no unproved placeholders.

References

[1] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. *NeurIPS*, 2017; arXiv:1706.08840.

[2] M. Farajtabar, N. Azizan, A. Mott, and A. Li. Orthogonal gradient descent for continual learning. *AISTATS*, PMLR 108, 2020; arXiv:1910.07104.

[3] Y. Sun et al. Learning to (learn at test time): RNNs with expressive hidden states. arXiv:2407.04620, 2024.

[4] B. Liu et al. Longhorn: State space models are amortized online learners. arXiv:2407.14207, 2024.

[5] S. Yang et al. Gated delta networks: Improving Mamba2 with the delta rule. arXiv:2412.06464, 2024.

[6] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. *ICML*, 2020; arXiv:2006.16236.

[7] J. von Oswald et al. Transformers learn in-context by gradient descent. *ICML*, 2023; arXiv:2212.07677.

[8] D. Dai et al. Why can GPT learn in-context? Language models implicitly perform gradient descent as meta-optimizers. arXiv:2212.10559, 2022.

[9] R. Hendel, M. Geva, and A. Globerson. In-context learning creates task vectors. *EMNLP Findings*, 2023; arXiv:2310.15916.

[10] E. Todd et al. Function vectors in large language models. arXiv:2310.15213, 2023.